

Intrabar Order Pressure

```
//@version=6
```

```
indicator("Intrabar Order Pressure", overlay=true, max_bars_back=500, max_labels_count=500)
```

```
// — UDT
```

```
type LabelStore
```

```
    array<label> labels = na
```

```
type Trade
```

```
    varip float price = na
```

```
    varip float delta = na
```

```
type TradeList
```

```
    varip array<Trade> trades
```

```
type TradeStore
```

```
    varip map<int, TradeList> data
```

```
// — Inputs
```

```
float minLots   = input.float(150, "Minimum lots threshold", minval=0.0001)
```

```
int  maxCandles = input.int(5, "Max candles to display", minval=1, maxval=20)
```

```
color buyColor  = input.color(color.new(color.green, 50), "Buy color")
```

```
color sellColor = input.color(color.new(color.red, 50), "Sell color")
```

```
// — Persistent state
```

```
varip TradeStore store = TradeStore.new(map.new<int, TradeList>())
```

```
varip float      prevVolume = na
```

```
varip float      prevPrice  = close
```

```
varip int         currentIdx = na
```

```
var  map<int, LabelStore> labels = map.new<int, LabelStore>()
```

// ——— Helpers

```
f_lotSize(float lots) =>
  lots >= minLots * 6 ? size.huge :
  lots >= minLots * 4 ? size.large :
  lots >= minLots * 2 ? size.normal : size.small
```

// ——— On every realtime tick

if barstate.isrealtime

// 1. Reset prevVolume on new bar to avoid cross-bar delta bleed

if na(currentIdx) or currentIdx < bar_index

log.info("New candle")

prevVolume := na

currentIdx := bar_index

// 2. Compute volume delta

float delta = na(prevVolume) ? na : (volume - prevVolume) * (close > prevPrice ? 1 : -1)

prevVolume := not na(volume) ? volume : prevVolume

// 3. Record trade if above threshold

TradeList candle = map.get(store.data, currentIdx)

if na(candle)

candle := TradeList.new(array.new<Trade>())

map.put(store.data, currentIdx, candle)

if not na(delta) and math.abs(delta) >= minLots

log.info(str.toString(delta) + ", " + str.toString(prevVolume))

if not map.contains(store.data, currentIdx)

map.put(store.data, currentIdx, TradeList.new(array.new<Trade>()))

Trade trade = Trade.new(

```

    close,
    delta
)
array.push(candle.trades, trade)

```

```

// 4. Draw labels for real-time candle without persistence

```

```

for tradeN in candle.trades

```

```

    float atPrice = tradeN.price

```

```

    float withDelta = tradeN×delta

```

```

    color col      = withDelta >= 0 ? buyColor : sellColor

```

```

    label lbl      = label.new(

```

```

        x          = currentIdx,

```

```

        y          = atPrice,

```

```

        text       = "●",

```

```

        textcolor  = col,

```

```

        color      = na,

```

```

        style      = label.style_label_center,

```

```

        size       = f_lotSize(withDelta < 0 ? withDelta * -1.0 : withDelta),

```

```

        xloc       = xloc.bar_index,

```

```

        yloc       = yloc.price

```

```

    )

```

```

prevPrice := close

```

```

else

```

```

// 1. Check if values exist for candle

```

```

TradeList candle = map.get(store.data, bar_index)

```

```

if not na(candle)

```

```

    // 2. If yes, create persisted labels for candle

```

```

    if not map.contains(labels, bar_index)

```

```

        // Create labels

```

```

        LabelStore candleLabels = LabelStore.new()

```

```

        for trade in candle.trades

```

```

            float atPrice = tradexprice

```

```

            float withDelta = tradexdelta

```

```

            color col      = withDelta >= 0 ? buyColor : sellColor

```

```

            label lbl      = label.new(

```

```

x      = bar_index,
y      = atPrice,
text   = "●",
textcolor = col,
color  = na,
style  = label.style_label_center,
size   = f_lotSize(withDelta < 0 ? withDelta * -1.0 : withDelta),
xloc   = xloc.bar_index,
yloc   = yloc.price
)
array.push(candleLabels.labels, lbl)

```

```
map.put(labels, bar_index, candleLabels)
```

```
// Remove excessive labels
```

```
if bar_index > maxCandles
```

```
    int clearIdx = bar_index - maxCandles
```

```
    if map.contains(store.data, clearIdx)
```

```
        map.remove(store.data, clearIdx)
```

```
    if map.contains(labels, clearIdx)
```

```
        LabelStore clearLabels = map.get(labels, bar_index)
```

```
        for clearLabel in clearLabels.labels
```

```
            label.delete(clearLabel)
```